



Security Assessment Report



DelegationManager Upgrade PR #84

June 2026

Prepared for EigenLayer

Table of Contents

Project Summary	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology	4
Findings Summary.....	5
Severity Matrix.....	5
Detailed Findings	6
Informational Severity Issues.....	7
I-01: Invariant breaks for beaconChainETHStrategy resulting in over-accounting.....	7
Observations	8
Disclaimer	10
About Certora	10

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
EigenLayer – DelegationManager Upgrade – PR #84	https://github.com/Layr-Labs/eigenlayer-contracts-private/pull/84	b2eebba	Solidity / EVM

Project Overview

This document describes the security review of **EigenLayer – DelegationManager Upgrade – PR #84**. The work was undertaken from **June 29th, 2026** to **July 1st, 2026**.

The following contract list is included in our scope:

`src/contracts/**/*.sol` (changes introduced/affected by PR #84)

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

EigenLayer is an Ethereum-based protocol that introduces the mechanism of restaking, allowing users to repurpose their already staked ETH or Liquid Staking Tokens (LSTs). By opting into EigenLayer's smart contracts, stakers can extend cryptoeconomic security to additional external services, known as Actively Validated Services (AVSs), without needing to lock up new assets. This shared security model enables these new applications to efficiently bootstrap trust while providing restakers with opportunities to earn supplementary rewards.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	–	–	–
Informational	1	1	0
Total	1	1	0

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
I-01	Invariant breaks for beaconChainETHStrategy resulting in over-accounting	Informational	Acknowledged

Informational Severity Issues

I-01: Invariant breaks for beaconChainETHStrategy resulting in over-accounting

Files:

src/contracts/core/DelegationManager.sol

Description:

The PR #84 upgrade introduces a core invariant expected to hold true: `queueSlashable shares <= removedFromOperator shares`. While this invariant holds for ERC20 strategies, it currently breaks for the `beaconChainETHStrategy`.

Specifically, on the beacon path, the queued value is recorded by dividing `withdrawableShares` by the full slashing factor (`operator maxMagnitude * beaconChainSlashingFactor`), but `_getSlashableSharesInQueue()` later derives the queued slashable amount by multiplying only by `prevMaxMagnitude`. Consequently, when the beacon chain slashing factor is below 1, the queued slashable shares can end up being larger than the shares actually removed from `operatorShares`. While PR #84 did not introduce this directly, and there is no immediate impact since `burnableETHShares` is practically unused, the invariant is nonetheless broken.

Recommendations:

Acknowledge this edge case. If `burnableETHShares` functionality is fully utilized in the future, the math on the beacon chain path will need to be refactored to respect the queue slashable invariant.

Customer Response:

Acknowledged. – “We are aware of this edgecase and the burning functionality for beacon shares will remain disabled so we don't see any way how this may become exploitable.”

Observations

The following notes were shared directly with the EigenLayer team during the review process regarding the PR #84 upgrade:

- The change looks directionally correct to us. `_removeSharesAndQueueWithdrawal()` is changing so that the amount added to queued slashable accounting is derived from the actual `withdrawableShares` removed from `operatorShares`, and not from the raw `scaledShares` stored in the withdrawal. This makes the queue shares increase aligned with the operator share decrease, and prevents the cases where rounding could add shares to the slashable queue without removing the corresponding amount from `operatorShares`. The tests added also make it more clear, especially the `test_slashOperatorShares_DoesNotDoubleCountRoundedQueueDust`, which shows slash → queue withdrawal → slash and that the previous accounting could count the same rounded-down shares again in the second slash.
- Additionally, the upgrade is not a retroactive fix (it would be hard to implement it in any case) so after the upgrade we may see `_cumulativeScaledSharesHistory` mixing shares that got calculated with the old way (`scaledShares`) added with shares got calculated with the new way (`withdrawableShares / slashingFactor`). This doesn't introduce new risk but it also means that until `MIN_WITHDRAWAL_DELAY_BLOCKS` has elapsed from the last pre upgrade queue event, we may still see over accounting. Pausing queuing/slashings for this is an option but wouldn't recommend it because we think it's too strict.
- Before the change there was this edge case of over-slashing, so we examined if now we can have under-slashing and whether stakers can take advantage of it. We concluded that this is not exploitable because if `withdrawableShares` rounds down when the withdrawal is queued, then the `sharesToWithdraw` at completion is computed from the same rounded `scaledShares * slashingFactor` path, so the staker's actual withdrawal rounds down as well. In other words, the new `slashableScaledShares` calculation does not let the staker withdraw value that was skipped upon queuing.

- We also reviewed the upgrade scripts for the new **DelegationManager** implementation and they look good. They follow the same structure as the previous release upgrade scripts and there are no issues in them.

Overall, we think the change is good.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.